VOLUME III · THE PAST-PAPER GAPS · EVERYTHING THE CLASS NEVER REACHED

CMS 706 — ER Diagrams, Query Processing & the Rest of the Paper

Volume I covers the 29 pages of class notes. But the lecturer's own paper of 14/07/2026 devotes **four of its seven questions** to material the notes never touched — ER diagrams, the phases of query processing, the phases of data design and division-style queries — and the older CMS 445 paper adds functional dependency, normalization and distributed databases. This volume is all of that, worked.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Friday 07 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer

WHY THIS VOLUME MATTERS MORE THAN A NORMAL GAP-FILLER

Both past papers say "**attempt any FIVE**". On the lecturer's 2026 paper, only questions **1, 2 and 3** can be answered from the class notes alone. To reach five answers you **must** take at least two of questions 4–7, and every one of those four rests on material below. **ER diagrams alone are a 14-mark question.** Learn Volume I first, then §1, §2 and §3 here — in that order.

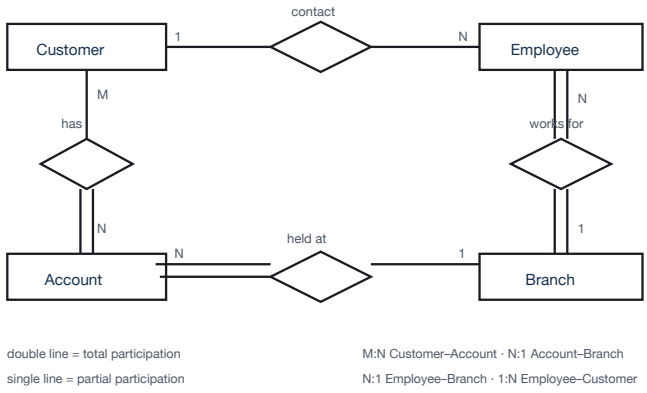
1 ER modelling — the 14-mark question

An **Entity–Relationship (ER) model** is a **conceptual, graphical model of the data requirements of a system**, showing the **entities** about which data is stored, the **attributes** that describe them and the **relationships** among them. It is produced at the **conceptual design** phase (§3) and is later mapped to a relational schema.

THE NOTATION YOU MUST DRAW

SYMBOL	MEANING
Rectangle	An **entity** — a thing about which data is stored (Customer, Account)
Double rectangle	A **weak entity** — cannot be identified without an owner entity
Ellipse	An **attribute**
Underlined ellipse	A **key attribute** — uniquely identifies the entity
Double ellipse	A **multivalued attribute** — e.g. several phone numbers
Dashed ellipse	A **derived attribute** — computed, e.g. age from date of birth
Diamond	A **relationship** between entities
Line with 1 / N / M	The **cardinality** — 1:1, 1:N or M:N
Double line	**Total participation** — every instance must take part
Single line	**Partial participation** — participation is optional

THE BANK ER DIAGRAM, AS SET IN THE PAST PAPER



Entities, relationships and cardinalities. Attributes are listed in the table below — in the exam draw them as ellipses hanging off each rectangle.

ENTITY	KEY ATTRIBUTE	OTHER ATTRIBUTES
Customer	SSN	name · **phone (multivalued — double ellipse)** · occupation
Account	account-no	balance · type (e.g. savings)
Branch	branch-code	location · number-of-employees
Employee	SSN	name · salary

HOW TO READ A REQUIREMENT INTO CARDINALITY

THE WORDING	WHAT IT MEANS
"belongs to **exactly one**"	Cardinality **1** on that side, and **total** participation (double line)
"can have **any number** of"	Cardinality **N**, partial participation
"**one or more**"	Cardinality **N**, **total** participation
"**zero or more**"	Cardinality **N**, **partial** participation
"**at most one**"	Cardinality **1**, **partial** participation
"are **not required** to have"	Explicitly **partial** — this phrasing is a deliberate hint

Requirement → diagram, line by line. Customer — key `SSN`, attributes `name`, `phone` (**double ellipse**, "one or more"), `occupation`. **Account** — key `account-no`, attributes `balance`, `type`. **Customer–Account is M:N** ("an account belongs to one or more customers; a customer can have any number of accounts") with **total participation on the account side**. **Account–Branch is N:1**, total on the account side ("exactly one branch") and **partial on the branch side** ("branches are not required to have accounts"). **Branch** — key `branch-code`, attributes `location`, `no-of-employees`. **Employee** — key `SSN`, attributes `name`, `salary`. **Employee–Branch is N:1**, total both ways ("works for exactly one branch"; "branches have one or more employees"). **Employee–Customer is 1:N contact**, partial ("zero or more customers"; "at most one employee as a contact").

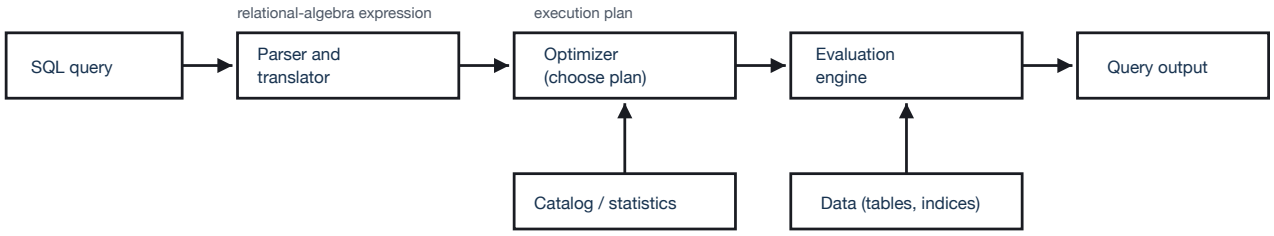
Mapping an ER model to relations — do this if asked, or to check yourself

1. **Each entity → a table**, with the key attribute as the primary key.
2. **Each multivalued attribute → its own table**, keyed on (owner key + the value). Phone numbers cannot live in the Customer row.
3. **1:N relationship → post the key of the "1" side into the "N" side** as a foreign key.
4. **M:N relationship → a new table** holding both keys as a composite primary key.
5. **1:1 relationship** → post either key into the other, usually into the side with total participation.

```
Customer(SSN, name, occupation, contact_emp_SSN)
CustomerPhone(SSN, phone)        ← multivalued
Account(acct_no, balance, type, branch_code)
Holds(SSN, acct_no)                ← M:N
Branch(branch_code, location, no_of_emps)
Employee(SSN, name, salary, branch_code)
```

2 Phases of query processing — the 8-mark diagram question

Query processing is the set of activities a DBMS performs to **translate a high-level query into an efficient sequence of low-level operations and execute it** against the stored data.



Draw this shape and label every box and arrow — the marks are for the stages and their outputs, not for artistry.

#	STAGE	FUNCTION	OUTPUT IT HANDS ON
1	**Parsing and translation**	Checks the query's **syntax**, verifies that the named **relations and attributes actually exist** in the data catalog, checks the user's **access rights**, and translates the query into an internal **relational-algebra** expression	A parse tree / relational-algebra expression
2	**Optimization**	Generates the **equivalent alternative execution plans** and estimates the cost of each using the **catalog statistics** — table sizes, index availability, selectivity — then **chooses the cheapest**	The chosen **query execution plan**
3	**Evaluation / execution**	The **evaluation engine** runs the chosen plan against the actual stored data, applying the physical operators — index scan, table scan, join, sort, aggregate	The **query result** returned to the user

If more stages are wanted, split stage 1 into *parsing* (syntax) and *translation* (to relational algebra), and add a final *result return* stage — a five-stage answer of parsing → translation → optimization → evaluation → output is equally acceptable, and it reads better against an 8-mark allocation. Note the tie back to Volume I §6: **the optimizer chooses the best execution plan by reducing disk access, reducing CPU usage, and using indexes and statistics** — that sentence from the notes *is* stage 2.

Debugging the two broken queries PAST Q · 6 MARKS

(I) WHY IS THIS QUERY REJECTED?

```
BusDriver(bdno varchar2(5),
          bdname varchar2(50), … )

select d_no from BusDriver where d_no = 07;
```

The fault: the attribute `d_no` **does not exist** in the relation `BusDriver` — the key column is called `bdno`. The query is therefore rejected at the very first stage of query processing, **parsing and translation**, when the parser consults the **data catalog** and cannot resolve the identifier. It never reaches the optimizer.

Secondary fault worth a mark: even with the right name, `bdno` is declared `varchar2(5)` — a **character** type — so comparing it with the **numeric** literal `07` is a type mismatch. A character comparison would also lose the leading zero.

(II) WHY IS THIS QUERY INCORRECT?

```
BusDriver(bdno, bdname, bdsalary, dno)
Depot(dno, dname, daddress)

select bdno, bdname from BusDriver, Depot
where bdsalary > 5000
  and dname = 'Hornsey'
  and BusDriver.dno = Depot.dno
  and bdsalary < 2000;
```

The fault: the WHERE clause is **self-contradictory** — it requires `bdsalary > 5000` **and** `bdsalary < 2000` at the same time. No value can satisfy both, so the predicate is **unsatisfiable** and the query **always returns the empty set**. It is syntactically valid and will not be rejected — it simply cannot ever be right, which makes it worse than an error.

Solution
```
select bdno from BusDriver where bdno = '07';
```

Solution — decide which was meant, e.g.
```
select bdno, bdname
from   BusDriver, Depot
where  bdsalary > 5000
  and  dname = 'Hornsey'
  and  BusDriver.dno = Depot.dno;
```

If the intent was a band, use `bdsalary between 2000 and 5000`.

3 Phases of data design — the work-plan question PAST Q · 8 MARKS

The question asks for a **work plan** giving, for each phase: **a description · the inputs · the outputs · a key issue addressed**. Answer it as a table — it is faster to write and impossible to mark down for missing a part.

PHASE	DESCRIPTION	INPUTS	OUTPUTS	KEY ISSUE ADDRESSED
1. Requirements analysis	Gather and document what data the business needs to store and what it must do with it, by interviewing users and studying existing documents and systems	Interviews, existing forms and reports, business rules, legacy files	A written **requirements specification** — data items, volumes, users, rules	**Completeness and ambiguity** — have all user groups been consulted, and do any two of them contradict each other?
2. Conceptual design	Build a **DBMS-independent** model of the data: identify entities, attributes and relationships	The requirements specification	A **conceptual schema** — the **ER model / ER diagram**	**Correct entities and cardinalities** — is this really an entity or just an attribute, and is that relationship 1:N or M:N?
3. Logical design	Map the conceptual model onto the **data model of the chosen DBMS** — for an RDBMS, turn entities and relationships into tables, keys and foreign keys, then **normalize**	The ER model, plus the choice of DBMS	A **logical schema** — normalized relations with primary and foreign keys and integrity constraints	**Redundancy and the update anomalies** — normalization to remove insert, update and delete anomalies (§5)
4. Physical design	Decide **how** the logical schema will actually be stored and accessed on the chosen hardware	The logical schema, expected query workload, volumes, hardware	Storage structures, **indexes**, file organization, partitioning, tuning parameters	**Performance vs storage trade-off** — every index speeds reads but costs space and slows writes (Vol I §5)
5. Implementation and testing	Create the database with DDL, load the data, and validate it against the requirements	The physical design and the source data	A **working, populated database** plus test results and documentation	**Data migration and validation** — is legacy data clean and correctly converted?

How to frame the answer for the inventory-tracking scenario. Open with one sentence: "*The data design team's work plan consists of five phases, each with defined inputs and deliverables, listed below.*" Then the table. Then close by tying it to what the manager actually asked for: the **phase boundaries are what let the overall timeframe and cost be estimated**, because each phase's output is a reviewable deliverable — the requirements specification, the ER diagram, the normalized schema, the physical design, and the tested database. Naming a deliverable per phase is what turns a list of activities into a work plan.

4 The committee queries — division and its cousins PAST Q · 9 MARKS

```
professor(profname, deptname)      department(deptname, building)      committee(profname, commname)
```

(I) IN ANY ONE OF SMITH'S COMMITTEES — 2 MARKS

```
SELECT DISTINCT c.profname
FROM   committee c
WHERE  c.commname IN
       (SELECT commname
        FROM   committee
        WHERE  profname = 'Smith');
```

"Any one" means **intersection is non-empty** — a simple `IN` subquery. Add `AND c.profname <> 'Smith'` if you want to exclude Smith himself; say which reading you chose.

(IV) OFFICE IN THE ICICS BUILDING — 2 MARKS

```
SELECT p.profname
FROM   professor p, department d
WHERE  p.deptname = d.deptname
  AND  d.building = 'ICICS';
```

The only trick is the sentence "*a professor's office is in the building in which her/his department is in*" — it tells you to **join through** `department` rather than look for a building column on `professor`.

(II) IN AT LEAST ALL OF SMITH'S COMMITTEES — 2 MARKS

This is **relational division**: professors for whom **there is no committee of Smith's that they are missing**. The standard SQL idiom is the **double NOT EXISTS**.

```
SELECT DISTINCT p.profname
FROM   committee p
WHERE  NOT EXISTS (
   SELECT s.commname FROM committee s
   WHERE  s.profname = 'Smith'
   AND    NOT EXISTS (
      SELECT * FROM committee x
      WHERE  x.profname = p.profname
      AND    x.commname = s.commname));
```

Counting alternative, which is easier to write correctly under pressure:

```
SELECT c.profname FROM committee c
WHERE  c.commname IN (SELECT commname FROM committee
                      WHERE profname='Smith')
GROUP BY c.profname
HAVING COUNT(DISTINCT c.commname) =
       (SELECT COUNT(*) FROM committee
        WHERE profname = 'Smith');
```

(III) IN EXACTLY SMITH'S COMMITTEES — 3 MARKS

"No more and no less" = **at least all of them** (ii) **and no others**. Easiest as two counts that must agree:

```
SELECT c.profname FROM committee c
GROUP BY c.profname
HAVING COUNT(*) = (SELECT COUNT(*) FROM committee
                   WHERE profname = 'Smith')
   AND COUNT(*) = (SELECT COUNT(*) FROM committee c2
                   WHERE c2.profname = c.profname
                   AND c2.commname IN
                       (SELECT commname FROM committee
                        WHERE profname = 'Smith'));
```

In words: the professor sits on **the same number** of committees as Smith, **and every one of those is one of Smith's**.

> **The three-word recognition rule.** "**any one of**" → `IN` subquery. "**at least all**" → **division**, so double `NOT EXISTS` or a `HAVING COUNT` comparison. "**exactly**" → division **plus** a count equality that forbids extras. Spotting which of the three you have been given is most of the mark.

5 Functional dependency, anomalies and normalization PAPER A

A functional dependency `X → Y` exists when, **for any two rows that agree on X, they must also agree on Y** — the value of X **determines** the value of Y. X is the **determinant**.

TYPE	DEFINITION AND EXAMPLE
Full functional dependency	Y depends on **the whole** of a composite X and not on any part of it. `(StudentID, CourseID) → Grade`
Partial dependency	Y depends on **part** of a composite key. `(StudentID, CourseID) → StudentName` is partial, since `StudentID → StudentName` alone. Removed by **2NF**
Transitive dependency	X → Y and Y → Z, so X → Z **indirectly**. `StudentID → DeptID → DeptName`. Removed by **3NF**
Trivial dependency	Y is a **subset** of X, so it holds automatically. `(StudentID, Name) → Name`
Multivalued dependency	X determines a **set** of Y values independently of other attributes. Removed by **4NF**

Three advantages of functional dependency: it is the basis of **normalization** and so removes redundancy · it lets **keys and integrity constraints** be identified formally · it preserves **data consistency** by making the relationships between attributes explicit.

THE THREE ANOMALIES — ALWAYS EXPLAIN WITH ONE TABLE

Take an unnormalized `StudentCourse(StudentID, StudentName, DeptName, CourseID, Grade)`:

ANOMALY	WHAT GOES WRONG
Insert anomaly	You **cannot record a new department** until some student enrols in it, because the primary key requires a student — data you have cannot be stored
Update anomaly	A student's name appears in **every row** for every course they take. Changing it means changing many rows, and missing one leaves the database **inconsistent**
Delete anomaly	Deleting the **last** student in a department **destroys the department's data too** — you lose facts you meant to keep

THE NORMAL FORMS, IN ONE LINE EACH

FORM	REQUIREMENT
1NF	All attribute values are **atomic** — no repeating groups or multivalued cells
2NF	1NF **and** no **partial** dependency on part of a composite key
3NF	2NF **and** no **transitive** dependency on the key
BCNF	Every determinant is a **candidate key**

Normalization is the process of organizing tables to **reduce redundancy and eliminate the three anomalies**, by decomposing one wide table into several related ones joined on keys.

6 Transactions, concurrency and recovery BOTH PAPERS

TRANSACTION — THE DEFINITION AND ITS DELIMITERS

A **transaction** is a **sequence of actions that represent a logical unit of work**, transforming the database from one consistent state to another. It is **scoped by the delimiters** `BEGIN TRANSACTION` … `COMMIT` **or** `ROLLBACK`. After successful execution the transaction reaches the **partially committed** state, and once **committed** the changes are **permanent and durable** — the point of no return, after which recovery will redo rather than undo them.

ACID — THE FOUR PROPERTIES

PROPERTY	MEANING
Atomicity	**All or nothing** — either every action happens or none does

CONCURRENT EXECUTION AND WHY CONTROL IS NEEDED PAST Q

Concurrent execution means several transactions from different users are **interleaved and executed in overlapping time** in a multiuser system, rather than one strictly after another. It is done for **throughput and resource utilisation** — the CPU can work on one transaction while another waits for disk — and for **reduced waiting time**, so a short transaction is not stuck behind a long one.

Concurrency control is needed because uncontrolled interleaving destroys consistency. The four classic problems:

PROPERTY	MEANING
Consistency	The database moves from one **valid state** to another, respecting all constraints
Isolation	Concurrent transactions do not interfere; each sees the database as if **alone**
Durability	Once committed, changes **survive failure**

TRANSACTION OPERATIONS AND STATES

Operations (4): `READ` · `WRITE` · `COMMIT` · `ROLLBACK` (also `BEGIN`).
States (5): Active → **Partially committed** (last statement executed) → **Committed** (changes permanent); or **Failed** → **Aborted** (rolled back to the original state).

PROBLEM	WHAT HAPPENS
Lost update	Two transactions read the same value and both write; the second overwrites the first, whose update vanishes
Dirty read	One transaction reads a value written by another that then **aborts** — it read data that never officially existed
Unrepeatable read	A transaction reads the same row twice and gets **different values**, because another committed in between
Phantom read	A repeated query returns **extra rows** inserted by another transaction

Quote the notes' own sentence as the definition: concurrency control is the DBMS activity of **coordinating processes that operate concurrently, access shared data and can potentially interfere with one another; its goal is to allow concurrency while maintaining the consistency of the shared data**.

LOCKING — PESSIMISTIC VS OPTIMISTIC PAPER A

APPROACH	HOW IT WORKS
Pessimistic locking	Assumes conflict is **likely**: a transaction **acquires a lock before** touching data and holds it, so others must wait. **Shared (read) locks** may be held by many; an **exclusive (write) lock** by one only. Safe, but risks **waiting and deadlock**
Optimistic locking	Assumes conflict is **rare**: transactions proceed with **no locks**, and at commit time the system **validates** that nothing they read was changed meanwhile. If it was, the transaction is **rolled back and retried**. Fast when conflicts are rare, wasteful when they are not

Two-phase locking (2PL) is the standard protocol: a **growing phase** where locks are only acquired, then a **shrinking phase** where they are only released. It guarantees serializability.

COMMIT AND TWO-PHASE COMMIT

Commit makes a transaction's changes **permanent** in a single database. **Two-phase commit (2PC)** is the protocol used when a transaction spans **several databases or sites**, so all of them must agree:

1. **Phase 1 — prepare/voting.** The coordinator asks every participant "can you commit?" Each writes its work to a log and replies **ready** or **abort**.
2. **Phase 2 — commit/decision.** If **all** voted ready, the coordinator broadcasts **commit** and all commit; if **any** voted abort, it broadcasts **abort** and all roll back.

The point: it preserves **atomicity across sites** — never some sites committed and others not. Its weakness is **blocking** if the coordinator fails between the phases.

7 Distributed databases PAPER A

A distributed database is a single logical database whose data is **physically spread across several sites connected by a network**, yet appears to the user as one database.

FOUR ADVANTAGES

1. **Reliability and availability** — one site failing does not take the whole system down
2. **Improved performance** — data is held near the users who use it, cutting network traffic and response time
3. **Modular growth** — new sites are added without redesigning the whole system
4. **Local autonomy** — each site controls its own data while still sharing it

FRAGMENTATION AND REPLICATION

Fragmentation is the process of **dividing a relation into smaller pieces (fragments) which are stored at different sites**. **Horizontal** fragmentation splits by **rows**, **vertical** by **columns**, and **mixed/hybrid** does both.

Two advantages of replication: availability — data survives a site failure; **faster local reads** — queries are answered from a nearby copy.
Two advantages of fragmentation: efficiency — only the relevant subset is stored and scanned at each site; **locality** — data lives where it is used, reducing network cost.

TRANSPARENCY — THE 7-MARK PART

Transparency means the distribution is **hidden from the user**, who queries the system as though it were a single ordinary database. Its kinds:

DISTRIBUTED VS CENTRALIZED — THREE AREAS

AREA	DISTRIBUTED	CENTRALIZED
Data location	Spread across many sites	All at one site
Reliability	High — no single point of failure	Low — one failure stops everything
Complexity & cost	High — needs replication, fragmentation, distributed control	Lower — simpler to design, secure and administer

THE TWO CLASSES

Homogeneous — all sites run the **same DBMS software** and are aware of each other. Two types: **autonomous** (sites operate independently) and **non-autonomous** (a central node coordinates).

Heterogeneous — sites run **different DBMS software** or data models, needing translation. Two types: **federated** (sites keep their independence and cooperate) and **multidatabase** (a layer presents one interface over them).

KIND	WHAT IS HIDDEN
Location transparency	**Where** the data physically resides
Fragmentation transparency	That a relation is **split into fragments** at all
Replication transparency	That **several copies** exist, and which one answered
Naming transparency	That names must be unique across sites
Transaction transparency	That a transaction touching several sites is coordinated (by 2PC)
Failure transparency	That a site has failed and another copy is serving

8 The remaining short gaps

PASSWORD POLICY — COMPLEXITY, FAILED ATTEMPTS, EXPIRY PAST Q · 5 MARKS

FEATURE	WHAT IT DOES AND HOW IT PROTECTS
Complexity	Requires a minimum length and a mix of upper case, lower case, digits and symbols, and forbids dictionary words or the username. **Protection:** it makes **brute-force and dictionary attacks** computationally infeasible by enlarging the search space
Failed attempts	Counts consecutive wrong passwords and **locks the account** after a threshold, for a delay or until an administrator releases it. **Protection:** it defeats **online guessing** outright, since an attacker gets only a handful of tries regardless of how weak the password is
Expired passwords	Forces a change after a set lifetime and blocks reuse of recent passwords. **Protection:** it **bounds the damage window** of a password that has been stolen or leaked without anyone noticing

Close by placing them: all three are **authentication** controls — the first of the six security controls in Volume I §7 — and they protect the database by keeping **unauthorized access** out before authorization and encryption ever come into play.

THE HOSPITAL DBMS SCENARIO PAST Q · 8 MARKS

(i) Hardware required: a database **server** with sufficient CPU and RAM · **storage** (RAID disks or SSD) sized for records and images · **backup storage** and an off-site copy · **client workstations** for wards and clinics · **networking** — switches, cabling, Wi-Fi for mobile devices · **UPS and power backup** · peripherals — scanners, barcode readers, printers.

Software required: a **DBMS** (Oracle, MySQL, SQL Server) · an **EHR application** front end · the **operating system** · **security software** — firewall, antivirus, encryption · **backup and recovery** software · reporting and analytics tools.

(ii) Benefits — quote Volume I: **reduced redundancy**, **data integrity**, **improved security**, **easy retrieval** and **multi-user access** (several clinicians reading one record at once), plus the sector benefits: **improved patient care, faster diagnosis, reduced errors** — and records that cannot be lost or misfiled as paper can.

A DATA CLEANING PLAN FOR MULTI-COUNTRY DATA PAST Q · 8 MARKS

"*A dataset from three different countries with varying languages, currencies and date formats.*" Give an ordered plan, naming the problem each step fixes:

1. **Profile and audit** the data first — record counts, column types, ranges, missing values and how each country's file differs.
2. **Standardize date formats** to one convention (ISO `YYYY-MM-DD`). **DD/MM vs MM/DD is silently destructive** — 03/08 is two different days — so convert using each source's known locale, not a guess.
3. **Normalize currency** — convert all monetary values to a single reporting currency at a **stated exchange rate and date**, and keep the original value and currency code in extra columns for audit.
4. **Resolve language and encoding** — set everything to UTF-8, transliterate or translate category labels to one language, and map synonyms onto one vocabulary.
5. **Deduplicate** — the same customer may appear once per country file; match on a stable key and merge.
6. **Handle missing values** — decide per column whether to drop, impute or flag, and document the choice.
7. **Correct data types and units** — numbers stored as text, decimal commas vs points, kg vs lb, and remove stray whitespace.
8. **Validate against rules** — ranges, formats, referential checks — then **document every transformation** so the cleaning is reproducible.

Tie it back: this is step 2, **data cleaning**, of the five-step data science process in Volume I §4 — "removing errors, duplicates, spaces".

DATABASES AND DATA SCIENCE — THE RELATIONSHIP

The relationship is **supply and consumption**. A database is the **organized, reliable store** where data is collected and kept with controlled redundancy and integrity; data science is the **discipline that extracts insight** from that data using algorithms and statistical methods. **Data science depends on databases** as its main source — the first of its five steps is "data is collected from databases, APIs and sensors" — and **without data, analysis is impossible**. Conversely, data science **feeds back into database design**: the queries analysts run drive which indexes are built, which is exactly the physical-design phase in §3. In short: **databases answer "what is stored"; data science answers "what does it mean"**.

IMPORTANCE OF DATA AS A DATABASE COMPONENT

Of the four components — data, relationship, constraints, schema — **data is the substance and the other three exist**

(iii) Implementation challenges — **high cost** of hardware, licences and expertise · **data migration** of years of paper records, which must be digitised and validated · **staff training and resistance to change** · **data privacy and regulatory compliance** (HIPAA and local law) · **integration** with lab and imaging systems · **downtime and continuity** — a hospital cannot stop · ongoing **maintenance and the need for a database administrator**.

only to serve it. Relationships describe correspondences *between data elements*; constraints are predicates defining the correct state *of the data*; the schema describes how *the data* is organized. Without data the database is an empty structure of no value: it is what users query, what decisions are made from, and what the whole DBMS apparatus of storage, security, concurrency and recovery is built to protect.

Two definitions the older paper asks for outright

THE ER MODEL

A **conceptual, graphical data model** that represents the data requirements of a system as **entities**, the **attributes** describing them and the **relationships** among them. It is **DBMS-independent** and is the output of the conceptual design phase — the blueprint later mapped onto tables.

THE RELATIONAL MODEL

A data model that **stores data in the form of tables (relations)** of rows and columns, using **keys** to establish relationships between them. It requires **few assumptions about how data is related or extracted**, so the same database can be **viewed in different ways** and spread across several tables; each table corresponds to an entity and each row to an instance of that entity.

9 Coverage map — where every past-paper question is answered

PAPER	Q	TOPIC	ANSWERED IN
DTS 304 14/07/2026 *this lecturer*	1	Hospital DBMS — hardware, software, benefits, challenges · logical independence · data as a component · consent	Vol III §8 · Vol I §10, §8, §7
	2	Databases and data science · data masking · data vs information · the data lifecycle	Vol III §8 · **Vol I §7, §2, §3**
	3	Data cleaning plan · indexing at 20 M records · GPS and decision-making · classifying data formats	Vol III §8 · **Vol I §5, §11, §4**
	4	**ER diagram for a bank (14 marks)**	**Vol III §1**
	5	Committee queries (any one / at least all / exactly) · office building · password policy	**Vol III §4 · §8**
	6	**Phases of query processing** · debugging two SQL queries	**Vol III §2**
	7	**Phases of data design** · concurrent execution and concurrency control · define transaction	**Vol III §3 · §6**

PAPER	Q	TOPIC	ANSWERED IN
CMS 445 2022/2023	1	Five SQL queries on an EMP table	Vol II §3 — every one worked
	2	Transaction concept, delimiters, properties, operations	**Vol III §6**
	3	Functional dependency — definition, four types, three advantages	**Vol III §5**
	4	Insert/update/delete anomalies · replication and fragmentation · transparency	**Vol III §5 · §7**
	5	Distributed databases — advantages, comparison, homogeneous vs heterogeneous, fragmentation	**Vol III §7**
	6	Commit and two-phase commit · optimistic and pessimistic locking · transaction states	**Vol III §6**
	7	Define the ER model and the relational model	**Vol III §8** · Vol I §9